

EduChamp — Project Handoff & Continuity Document

Version: 1.0 · **Date:** May 27, 2026 · **Author:** Manus AI

Checkpoint: f4c66049 (Sprint 26 — Stripe Integration, PayPal/ACH, Schools Hero)

Live domains: `educhamp.app` , `www.educhamp.app` , `educhamp.manus.space`

Table of Contents

1. [Executive Summary](#)
2. [Product Overview & Target Users](#)
3. [MVP Scope & Implementation Status](#)
4. [System Architecture](#)
5. [Technology Stack](#)
6. [File & Folder Structure](#)
7. [Authentication & Authorization](#)
8. [Database Schema](#)
9. [API Structure \(tRPC Routers\)](#)
10. [Payment & Subscription Architecture](#)
11. [Coupon & Promotional Discount System](#)
12. [Parent/Student Onboarding Workflows](#)
13. [AI Tutor Architecture](#)
14. [Curriculum & Course Structure](#)
15. [Admin Portal & CMS](#)
16. [Notification Systems](#)
17. [Third-Party Integrations](#)
18. [Frontend Design System](#)
19. [State Management & Data Flow](#)
20. [Security & Compliance](#)
21. [Testing Strategy](#)
22. [Deployment & DevOps](#)

- 23. [Environment Variables](#)
 - 24. [Known Issues & Technical Debt](#)
 - 25. [Roadmap & Future Enhancements](#)
 - 26. [User Roles & RBAC](#)
 - 27. [Business Rules & Validation Logic](#)
 - 28. [Seed Data & Content Management](#)
 - 29. [Analytics & Reporting](#)
 - 30. [Change Log & Sprint History](#)
 - 31. [Developer Setup Guide](#)
-

1. Executive Summary

EduChamp is an AI-powered adaptive learning platform built for K–12 students in Texas, with a primary focus on Algebra I aligned to the Texas Essential Knowledge and Skills (TEKS) standards. The platform combines a structured curriculum browser, an AI tutor (EduBot), a diagnostic placement engine, per-skill mastery tracking, a quiz engine, and a parent/guardian monitoring dashboard into a single cohesive product.

The business model is a subscription SaaS platform with three tiers: a free-to-start tier, a Family Plan (19.99/~~month~~15.99/mo billed annually), and a Premium Family Plan (29.99/~~month~~23.99/mo billed annually). A fourth tier — ISD/School Licensing — targets school district administrators and is sold through a demo-request and sales pipeline rather than self-service checkout.

The platform is built on a React 19 + Express 4 + tRPC 11 + MySQL (TiDB) stack, deployed on the Manus cloud platform with Stripe as the primary payment processor. As of Sprint 26, the platform is in a production-ready state with Stripe products created, webhooks implemented, and 141 automated tests passing.

2. Product Overview & Target Users

EduChamp serves four distinct user personas, each with a dedicated interface and workflow.

Students (Grades 3–12) are the primary learners. They complete a diagnostic placement test, follow an adaptive curriculum path, interact with the AI tutor EduBot in multiple modes (teach, practice, quiz, exam review, remediation), and track their mastery across individual skills. Students can be enrolled by parents or can sign up independently and invite a parent afterward.

Parents and Guardians monitor one or more children's progress from a dedicated Parent Dashboard. They receive AI-generated learning summaries, set academic goals, invite co-parents or guardians, and

manage their subscription and billing. The Premium Family Plan supports up to three student accounts under one parent subscription.

School/District Administrators (ISD) are enterprise leads who submit demo requests through the landing page. Their inquiries are tracked in a CRM pipeline within the Admin Console. This tier is not yet self-service; it is managed through a sales workflow.

Platform Administrators access the full Admin Console, which includes user management, course and curriculum CMS, coupon management, subscription CRM, payment analytics, newsletter management, demo request pipeline, RBAC role management, platform settings, and audit logs.

3. MVP Scope & Implementation Status

The following table summarises all major feature areas and their current implementation status.

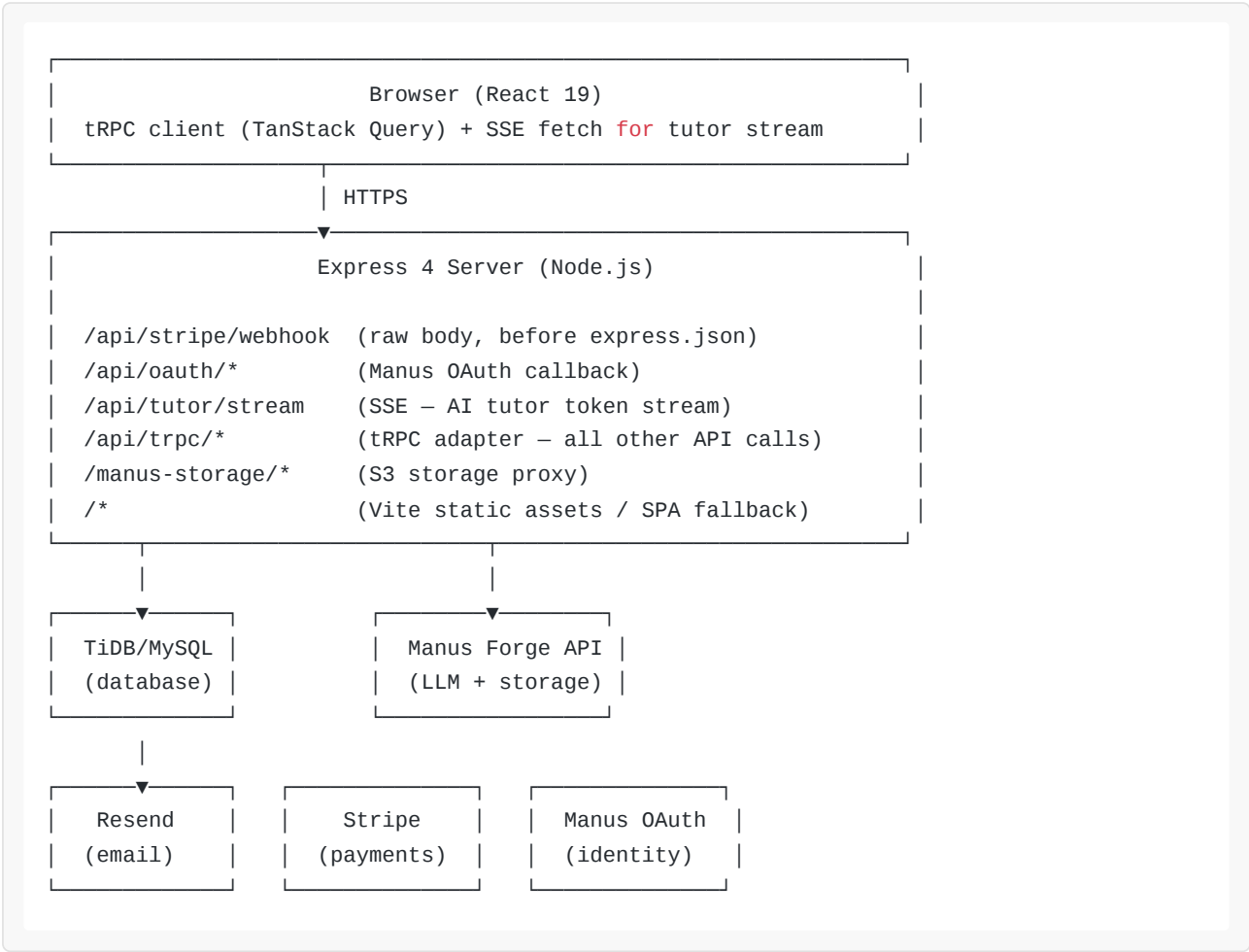
Feature Area	Status	Notes
Landing page with hero, features, pricing, FAQ	✓ Complete	Annual billing toggle, plan comparison table, Schools section
User authentication (Manus OAuth)	✓ Complete	JWT session cookie, protectedProcedure middleware
Student onboarding wizard	✓ Complete	3-step: profile → invite parent → complete
Parent onboarding wizard	✓ Complete	3-step: profile → AI goal alignment → complete
Parent–child account linking	✓ Complete	Invite tokens, co-parent access, enrolment invitations
Algebra I curriculum (Units 1–8)	✓ Complete	Lessons, worked examples, guided/independent problems
Diagnostic placement test	✓ Complete	Per-course, cooldown enforcement, unit-level results
Per-skill mastery tracking	✓ Complete	0–100 score, 5 mastery levels
Unit quiz engine	✓ Complete	Answer normalisation, 75% pass threshold, next-unit unlock
AI Tutor (EduBot) — streaming SSE	✓ Complete	6 modes, parent-summary mode, conversation history
Student dashboard	✓ Complete	Unit progress, mastery overview, recent quiz attempts
Parent dashboard	✓ Complete	Child selector, progress overview, AI summary
Course catalog (multi-course)	✓ Complete	56+ courses, grade-appropriate auto-enrollment
Admin Console — User Management	✓ Complete	CRUD, role/status/account-type, course enrollment
Admin Console — Course & Curriculum CMS	✓ Complete	Draft/publish workflow, version history
Admin Console — Platform Settings	✓ Complete	Key-value store, per-category grouping
Admin Console — RBAC	✓ Complete	Custom roles, granular permissions, role assignments

Feature Area	Status	Notes
Admin Console — Audit Log	✓ Complete	All admin actions recorded
Admin Console — Newsletter	✓ Complete	Subscriber management, AI-drafted campaigns
Admin Console — Demo Requests (CRM)	✓ Complete	12-field form, status pipeline, respond workflow
Admin Console — Coupon Manager	✓ Complete	Full CRUD, Stripe sync, redemption stats
Admin Console — Subscription CRM	✓ Complete	Paginated subscription table, per-user detail
Admin Console — Payment Analytics	✓ Complete	KPI cards, charts, CSV export
Stripe integration	✓ Complete	Products/prices created, checkout sessions, webhooks
PayPal / ACH payment methods	✓ Complete	Enabled in Stripe checkout session
Coupon redemption at checkout	✓ Complete	Real-time validation, Stripe coupon sync
Billing page (/billing)	✓ Complete	Subscription status, Stripe Customer Portal link
Annual billing toggle + 20% discount	✓ Complete	Persisted to users.billingPeriod
ISD demo request form	✓ Complete	12-field form, confirmation email, admin notification
Referral system	✓ Complete	Unique referral codes, reward tracking
Email notifications (Resend)	✓ Complete	Invite, confirmation, follow-up, newsletter
In-app notifications	✓ Complete	Per-user notification feed
Landing page AI chat	✓ Complete	Anonymous visitor chat, lead capture
Checkout success page	✓ Complete	Post-Stripe redirect landing

Feature Area	Status	Notes
Billing page	☒ Complete	Subscription status, payment history, portal link
Free trial period	☐ Planned	Sprint 27 — 7–14 day trial via Stripe
Subscription upgrade/downgrade flow	☐ Planned	Sprint 27 — Change Plan button on /billing
Auto-generated demo coupon for ISD leads	☐ Planned	Sprint 27
PayPal standalone flow	☐ Planned	Currently enabled via Stripe; standalone flow TBD
Mobile app (React Native)	☐ Future	Not scoped

4. System Architecture

EduChamp runs as a single Node.js process serving both the Express API and the Vite-built React frontend. In development, Vite runs as a middleware plugin within the Express server. In production, the Vite build outputs static assets that Express serves directly.



Key architectural decisions:

The tRPC-first approach means there are no manually written REST endpoints for application features. All client-server contracts are defined as typed procedures in `server/routers.ts` and its sub-routers, and consumed on the frontend with `trpc.*.useQuery/useMutation`. This eliminates the need for a separate API contract layer and ensures end-to-end type safety.

The AI tutor uses a dedicated SSE endpoint (`/api/tutor/stream`) rather than a tRPC procedure because tRPC does not natively support streaming responses. The endpoint authenticates via the same Manus SDK session cookie used by tRPC.

All file storage (images, documents) is handled through the Manus Forge S3-compatible storage layer via `server/storage.ts`. Files are never stored in the application directory or database columns.

5. Technology Stack

Layer	Technology	Version	Notes
Frontend framework	React	19.2.1	With concurrent features
Frontend routing	wouter	3.3.5	Lightweight SPA router
UI components	shadcn/ui + Radix UI	Various	Full component library
Styling	Tailwind CSS	4.1.14	OKLCH color tokens
Animation	Framer Motion	12.23.22	Page transitions, micro-interactions
Charts	Recharts	2.15.2	Admin analytics dashboards
Forms	react-hook-form + Zod	7.64 / 4.1.12	Type-safe form validation
Data fetching	TanStack Query	5.90.2	Via tRPC adapter
API layer	tRPC	11.6.0	End-to-end type safety
Serialisation	superjson	1.13.3	Date preservation across wire
Backend framework	Express	4.21.2	With Helmet, rate-limiting
Runtime	Node.js / tsx	22.x / 4.19.1	ESM modules
Database ORM	Drizzle ORM	0.44.5	MySQL dialect
Database	TiDB (MySQL-compatible)	Cloud	Managed, serverless
Auth	Manus OAuth 2.0	—	JWT session cookie (jose 6.1.0)
Email	Resend	6.12.4	Transactional + newsletter
Payments	Stripe	22.1.1	Subscriptions, webhooks
AI / LLM	Manus Forge API	—	GPT-4-class, streaming
File storage	Manus Forge S3	—	Via AWS SDK v3
Build tool	Vite	7.1.7	Dev middleware + production build
Testing	Vitest	2.1.4	141 tests across 7 test files
Package manager	pnpm	10.4.1	With workspace patches
TypeScript	TypeScript	5.9.3	Strict mode
Markdown rendering	streamdown	1.4.0	Streaming markdown in tutor UI

6. File & Folder Structure

```
/home/ubuntu/educhamp/
|
├─ client/                                ← React frontend (Vite)
|   ├── index.html                        ← Entry HTML, Google Fonts CDN
|   ├── public/                          ← Static config files only (favicon, robots.txt)
|   └─ src/
|       ├── _core/
|       │   ├── hooks/useAuth.ts         ← Auth state hook (reads trpc.auth.me)
|       │   └─ components/
|       │       ├── ui/                  ← shadcn/ui primitives (40+ components)
|       │       ├── AIChatBox.tsx        ← Reusable streaming chat component
|       │       ├── CheckoutModal.tsx    ← Stripe checkout with coupon entry
|       │       ├── DashboardLayout.tsx  ← Authenticated sidebar shell
|       │       ├── DashboardLayoutSkeleton.tsx
|       │       ├── ErrorBoundary.tsx
|       │       ├── ManusDialog.tsx
|       │       ├── Map.tsx              ← Google Maps proxy component
|       │       ├── RequestDemoModal.tsx ← ISD demo request 12-field form
|       │       ├── RoleSelectModal.tsx  ← Role selection during signup
|       │       └─ admin/
|       │           ├── CouponManagerTab.tsx
|       │           ├── DemoRequestsTab.tsx
|       │           ├── PaymentAnalyticsTab.tsx
|       │           └─ SubscriptionCRMTab.tsx
|       ├── contexts/
|       │   └─ ThemeContext.tsx
|       ├── hooks/
|       │   ├── useComposition.ts
|       │   ├── useMobile.tsx
|       │   └─ usePersistFn.ts
|       ├── lib/
|       │   ├── trpc.ts                  ← tRPC client binding
|       │   └─ utils.ts                  ← cn() utility
|       ├── pages/
|       │   ├── AdminDashboard.tsx       ← Full admin console (tabbed)
|       │   ├── Billing.tsx              ← /billing – subscription status + portal
|       │   ├── CheckoutSuccess.tsx      ← /checkout/success – post-Stripe landing
|       │   ├── CourseCatalog.tsx       ← Course browser
|       │   ├── CurriculumPage.tsx       ← Unit/lesson browser
|       │   ├── DiagnosticPage.tsx       ← Placement test UI
|       │   ├── Home.tsx                 ← Authenticated student home/dashboard
|       │   ├── LandingPage.tsx          ← Public marketing page
|       │   ├── NotFound.tsx
|       │   ├── ParentDashboard.tsx      ← Parent monitoring view
|       │   ├── ParentOnboarding.tsx     ← 3-step parent onboarding
|       │   ├── ProfilePage.tsx
|       │   ├── ProgressPage.tsx
|       │   ├── ReferralPage.tsx
|       │   ├── SkillsPage.tsx
|       │   └─ StudentOnboarding.tsx     ← 3-step student onboarding
```

└─ TutorPage.tsx	← AI tutor interface
└─ App.tsx	← Route map + layout wrappers
└─ index.css	← Global Tailwind + CSS variables
└─ main.tsx	← React root + providers
└─ drizzle/	
└─ schema.ts	← All 30+ table definitions
└─ relations.ts	← Drizzle relation definitions
└─ meta/	
└─ └─ _journal.json	← Migration history
└─ └─ migrations/	← Generated SQL migration files (0001-0021)
└─ server/	
└─ └─ _core/	
└─ └─ └─ context.ts	← tRPC context (user from cookie)
└─ └─ └─ cookies.ts	← Session cookie helpers
└─ └─ └─ dataApi.ts	← Manus Data API client
└─ └─ └─ env.ts	← Typed ENV object (all env vars)
└─ └─ └─ heartbeat.ts	← Scheduled job handler
└─ └─ └─ imageGeneration.ts	← Image generation helper
└─ └─ └─ index.ts	← Express bootstrap + route registration
└─ └─ └─ llm.ts	← invokeLLM() helper
└─ └─ └─ map.ts	← Google Maps proxy
└─ └─ └─ notification.ts	← notifyOwner() helper
└─ └─ └─ oauth.ts	← Manus OAuth flow
└─ └─ └─ sdk.ts	← Manus SDK (authenticateRequest)
└─ └─ └─ storageProxy.ts	← /manus-storage/* proxy
└─ └─ └─ systemRouter.ts	← system.notifyOwner procedure
└─ └─ └─ trpc.ts	← tRPC primitives + procedure types
└─ └─ └─ voiceTranscription.ts	← Whisper transcription helper
└─ └─ └─ vite.ts	← Vite dev middleware integration
└─ └─ routers/	
└─ └─ └─ admin.ts	← All admin procedures
└─ └─ └─ authEnhancements.ts	← 2FA, password reset
└─ └─ └─ coParent.ts	← Co-parent invite/access
└─ └─ └─ landing.ts	← Public landing + demo request
└─ └─ └─ newsletter.ts	← Newsletter subscriber + campaign
└─ └─ └─ onboarding.ts	← Student/parent onboarding + invites
└─ └─ └─ parent.ts	← Parent dashboard procedures
└─ └─ └─ parentTools.ts	← Parent utility procedures
└─ └─ └─ payment.ts	← Stripe checkout, coupons, subscriptions
└─ └─ └─ referral.ts	← Referral codes + rewards
└─ └─ auth.logout.test.ts	← Reference test file
└─ └─ db.ts	← All database query helpers
└─ └─ demoRequest.test.ts	← 33 tests for demo request procedures
└─ └─ educamp-helpers.ts	← getMasteryLevel, buildTutorSystemPrompt
└─ └─ educamp.test.ts	← 25 core curriculum/quiz/mastery tests
└─ └─ emailService.ts	← Resend wrapper with retry + audit log
└─ └─ payment.test.ts	← 36 payment/coupon/subscription tests
└─ └─ routers.ts	← appRouter composition + core procedures
└─ └─ storage.ts	← storagePut/storageGet S3 helpers
└─ └─ stripe.ts	← Stripe client, PLANS map, helpers
└─ └─ stripeWebhook.ts	← /api/stripe/webhook handler
└─ └─ tutorStream.ts	← /api/tutor/stream SSE handler

```

|
| └─ shared/
|   │ └─ _core/errors.ts      ← Shared error types
|   │ └─ const.ts           ← Shared constants
|   │ └─ types.ts           ← Shared TypeScript types
|
| └─ scripts/
|   │ └─ apply-migration-0020.mjs ← One-time migration helper
|   │ └─ create-stripe-products.mjs ← Creates Stripe products/prices (idempotent)
|
| └─ references/
|   │ └─ periodic-updates.md ← Heartbeat/scheduled job documentation
|
| └─ .project-config.json      ← Manus project metadata
| └─ components.json          ← shadcn/ui configuration
| └─ drizzle.config.ts        ← Drizzle Kit configuration
| └─ package.json             ← Dependencies + scripts
| └─ todo.md                  ← Feature/bug tracking (all sprints)
| └─ tsconfig.json            ← TypeScript configuration
| └─ vite.config.ts           ← Vite + Tailwind configuration
| └─ vitest.config.ts         ← Vitest configuration

```

Static assets (images, logos, media) are stored outside the project directory at `/home/ubuntu/webdev-static-assets/` and served via Manus CDN URLs returned by `manus-upload-file --webdev`. They must never be placed inside the project directory.

7. Authentication & Authorization

EduChamp uses **Manus OAuth 2.0** as its identity provider. There are no passwords stored in the application database — all identity is delegated to the Manus platform.

Authentication Flow

The sequence below describes what happens when an unauthenticated user clicks “Sign In”:

1. The frontend calls `getLoginUrl(returnPath?)` from `client/src/const.ts`, which encodes `window.location.origin` and the optional return path into a `state` parameter and redirects to the Manus OAuth portal (`VITE_OAUTH_PORTAL_URL`).
2. After the user authenticates on the Manus portal, the browser is redirected to `/api/oauth/callback?code=...&state=...`.
3. The server exchanges the code for a user profile via the Manus OAuth backend (`OAUTH_SERVER_URL`), creates or updates the user record in the `users` table, and issues a signed JWT session cookie (signed with `JWT_SECRET`, using `jose 6.1.0`).
4. The server reads the `origin` from the `state` parameter and redirects the browser back to the frontend at the correct URL.

5. Subsequent requests carry the session cookie. The tRPC context (`server/_core/context.ts`) decodes and verifies the JWT on every request, making `ctx.user` available to all procedures.

Procedure Authorization Levels

Four procedure types are defined in `server/_core/trpc.ts` :

Procedure Type	Guard	Use Case
<code>publicProcedure</code>	None	Landing page data, coupon validation, demo request submission
<code>protectedProcedure</code>	Valid session cookie	Any authenticated action (profile, progress, payment)
<code>studentProcedure</code>	Valid session + <code>accountType !== 'parent'</code>	Learning actions (mark lesson complete, submit quiz)
<code>adminProcedure</code>	Valid session + <code>role === 'admin'</code>	All Admin Console operations

Session Cookie

The session cookie is named `educhamp_session` (defined in `shared/const.ts`). It is `HttpOnly`, `SameSite=Lax`, and `Secure` in production. The cookie is cleared on logout via `trpc.auth.logout.useMutation()`.

8. Database Schema

The database is a TiDB (MySQL-compatible) cloud instance accessed via Drizzle ORM. All timestamps are stored as UTC. The schema is defined in `drizzle/schema.ts` and migrations are tracked in `drizzle/meta/_journal.json` (migrations 0001–0021 applied).

Core Tables

Table	Purpose	Key Columns
users	All user accounts	id, name, email, role (admin/user), accountType (student/parent), grade, school, billingPeriod, onboardingCompleted, stripeCustomerId
userProfiles	Extended demographic data	userId, dateOfBirth, gradeLevel, schoolType, schoolDistrict, parentGoalCategory, parentGoalDetail, parentSignupReason, onboardingStep, preferredName, aiWelcomeMessage
parentChildLinks	Parent–student relationships	parentId, childId, nickname, isActive, linkedAt
studentInviteTokens	Parent → child invite links	token, parentId, email, expiresAt, acceptedAt
parentInviteTokens	Student → parent invite links	token, studentId, parentEmail, parentPhone, status, expiresAt

Curriculum Tables

Table	Purpose	Key Columns
courses	Course catalogue	id, title, subject, gradeLevel, courseCode, teksCode, isDefault, isPublished, diagnosticCooldownDays
units	Units within a course	id, courseId, unitNumber, title, description, sortOrder
lessons	Lessons within a unit	id, unitId, title, content (JSON), sortOrder
skills	TEKS-aligned skills	id, skillId, name, description, unitId, courseId
quizQuestions	Quiz questions per unit	id, unitId, questionText, questionType, choices, correctAnswer, skillTag, difficulty
diagnosticQuestions	Diagnostic test questions	id, courseId, questionText, choices, correctAnswer, skillTag, unitNumber

Progress & Mastery Tables

Table	Purpose	Key Columns
userCourseEnrollments	Which courses a user is enrolled in	userId , courseId , isCurrent , enrolledAt
unitProgress	Per-unit progress per user	userId , unitId , unitNumber , status (locked/in_progress/quiz_unlocked/completed), lessonsCompleted , totalLessons , quizScore , quizAttempts
lessonProgress	Per-lesson completion	userId , lessonId , unitId , completed , completedAt
userMastery	Per-skill mastery score	userId , skillId , score (0–100), attemptCount , lastAttemptAt
quizAttempts	Quiz submission history	userId , unitId , unitNumber , score , totalQuestions , correctCount , answers (JSON), submittedAt
diagnosticAttempts	Diagnostic test submissions	userId , courseId , score , answers (JSON), unitResults (JSON), completedAt

AI Tutor Tables

Table	Purpose	Key Columns
tutorSessions	Conversation sessions	userId , unitId , lessonId , mode , messages (JSON array), createdAt , updatedAt

Communication & Notification Tables

Table	Purpose	Key Columns
emailLogs	All outbound email audit trail	to , subject , templateName , status , messageId , errorMessage
userNotifications	In-app notification feed	userId , type , title , message , isRead , metadata
newsletterSubscribers	Email newsletter list	email , name , source , isActive , subscribedAt
newsletterCampaigns	Newsletter campaigns	subject , content , audience , status , sentAt , recipientCount

CRM & Sales Tables

Table	Purpose	Key Columns
demoRequests	ISD/school licensing leads	fullName, schoolName, roleTitle, email, phone, numStudents, gradeLevels, subjects, interestType, status (new/contacted/demo_scheduled/proposal_sent/closed_won/closed_lost/on_hold), assignedTo, adminNotes

Payment & Subscription Tables

Table	Purpose	Key Columns
coupons	Promotional discount codes	code, discountType, discountValue, applicablePlans, eligibility, usageLimit, perUserLimit, usageCount, duration, expiresAt, status, stripeCouponId, stripePromotionCodeId
couponRedemptions	Coupon usage history	couponId, userId, planName, billingPeriod, originalAmountCents, discountAmountCents, finalAmountCents, stripeCheckoutSessionId
subscriptions	Active/cancelled subscriptions	userId, planName, billingPeriod, status, stripeCustomerId, stripeSubscriptionId, currentPeriodEnd, cancelAtPeriodEnd, trialEnd, amountCents
paymentAuditLog	Immutable payment event log	userId, event, stripeEventId, stripeObjectId, amountCents, currency, status, metadata

Admin & RBAC Tables

Table	Purpose	Key Columns
<code>rbacRoles</code>	Custom roles	<code>name</code> , <code>description</code> , <code>isDefault</code>
<code>rbacPermissions</code>	Granular permissions	<code>name</code> , <code>resource</code> , <code>action</code> , <code>description</code>
<code>rbacRolePermissions</code>	Role ↔ permission mapping	<code>roleId</code> , <code>permissionId</code>
<code>rbacUserRoles</code>	User ↔ role assignment	<code>userId</code> , <code>roleId</code>
<code>auditLogs</code>	Admin action log	<code>userId</code> , <code>action</code> , <code>resource</code> , <code>resourceId</code> , <code>details</code> , <code>ipAddress</code>
<code>platformSettings</code>	Key-value config store	<code>key</code> , <code>value</code> , <code>category</code> , <code>description</code>
<code>cmsDrafts</code>	CMS content drafts	<code>resourceType</code> , <code>resourceId</code> , <code>draftData</code> , <code>publishedAt</code> , <code>version</code>

9. API Structure (tRPC Routers)

All application API calls (except the AI tutor stream and Stripe webhook) go through the tRPC adapter at `/api/trpc`. The `appRouter` in `server/routers.ts` composes 11 sub-routers plus inline procedures.

Router Map

appRouter	
└─ auth	
└─ me	(public) → current user or null
└─ logout	(public) → clears session cookie
└─ curriculum	
└─ getUnits	(public) → all units
└─ getUnit	(public) → single unit by number
└─ getLessons	(public) → lessons for a unit
└─ getLesson	(public) → single lesson
└─ getSkillsByUnit	(public) → skills for a unit
└─ getAllSkills	(protected) → skills for active course
└─ progress	
└─ getDashboard	(protected) → full dashboard data
└─ getMastery	(protected) → all skills + mastery scores
└─ getAllCourseProgress	(protected) → all enrolled courses
└─ switchActiveCourse	(protected) → change active course
└─ getLessonProgress	(protected) → lesson completion for a unit
└─ markLessonComplete	(student) → mark lesson done + unlock quiz
└─ quiz	
└─ getQuestions	(protected) → quiz questions (no answers)
└─ submitQuiz	(student) → grade quiz + update mastery
└─ diagnostic	
└─ getQuestions	(protected) → diagnostic questions
└─ submit	(student) → save attempt + unlock all units
└─ getLatestAttempt	(protected) → most recent diagnostic
└─ getAllAttempts	(protected) → full diagnostic history
└─ tutor	
└─ getSession	(protected) → get/create tutor session
└─ updateSession	(protected) → append messages to session
[Streaming: POST /api/tutor/stream – SSE, not tRPC]	
└─ onboarding	
└─ getProfile	(protected)
└─ saveStudentProfile	(protected)
└─ saveParentProfile	(protected)
└─ generateGoalAlignment	(protected) → AI-generated goal statement
└─ completeOnboarding	(protected) → mark done + auto-enroll
└─ saveBillingPeriod	(protected) → persist billing period
└─ createStudentInviteToken	(protected) → parent invites child
└─ acceptStudentInvite	(public) → child accepts parent invite
└─ createParentInviteToken	(protected) → student invites parent
└─ acceptParentInvite	(public) → parent accepts student invite
└─ rejectParentInvite	(public)
└─ getPendingParentInvites	(protected)
└─ getPendingInvitesForEmail	(public) → check invites by email

└─ parent	
└─ getChildren	(protected) → linked children
└─ getChildProgress	(protected) → child's dashboard data
└─ getChildMastery	(protected)
└─ getAISummary	(protected) → LLM-generated progress summary
└─ coParent	
└─ invite	(protected) → invite co-parent
└─ accept	(public)
└─ list	(protected)
└─ authEnhancements	
└─ setup2FA	(protected)
└─ verify2FA	(protected)
└─ requestPasswordReset	(public)
└─ parentTools	
└─ [utility procedures for parent account management]	
└─ referral	
└─ getMyCode	(protected) → get/create referral code
└─ getReferralStats	(protected)
└─ applyReferral	(protected)
└─ landing	
└─ getStats	(public) → platform stats for landing page
└─ createChatSession	(public) → anonymous visitor chat
└─ chat	(public) → AI landing page chat
└─ submitDemoRequest	(public) → ISD lead capture
└─ newsletter	
└─ subscribe	(public)
└─ unsubscribe	(public)
└─ listSubscribers	(admin)
└─ createCampaign	(admin)
└─ aiDraftCampaign	(admin) → LLM-drafted newsletter
└─ markCampaignSent	(admin)
└─ payment	
└─ validateCoupon	(public) → check code validity + discount
└─ createCheckoutSession	(protected) → Stripe checkout URL
└─ getBillingPortalUrl	(protected) → Stripe Customer Portal
└─ getSubscriptionStatus	(protected)
└─ getPaymentHistory	(protected)
└─ saveBillingPeriod	(protected)
└─ admin.createCoupon	(admin)
└─ admin.listCoupons	(admin)
└─ admin.updateCoupon	(admin)
└─ admin.archiveCoupon	(admin)
└─ admin.getCouponStats	(admin)
└─ admin.listSubscriptions	(admin)
└─ admin.getSubscriptionDetail	(admin)
└─ admin.getPaymentAnalytics	(admin)

```

├─ admin
│   ├── getStats                (admin) → overview KPIs
│   ├── listUsers              (admin)
│   ├── updateUser             (admin)
│   ├── deleteUser             (admin)
│   ├── listCourses            (admin)
│   ├── createCourse           (admin)
│   ├── updateCourse           (admin)
│   ├── getPlatformSettings    (admin)
│   ├── updatePlatformSetting  (admin)
│   ├── getAuditLog            (admin)
│   ├── listDemoRequests       (admin)
│   ├── updateDemoRequest      (admin)
│   ├── respondToDemoRequest   (admin)
│   ├── listRoles              (admin)
│   ├── createRole             (admin)
│   ├── updateRole             (admin)
│   └─ [CMS draft/publish/revert/history]
└─ system
    └─ notifyOwner              (protected) → push notification to owner

```

Non-tRPC Endpoints

Route	Method	Auth	Purpose
/api/oauth/callback	GET	None	Manus OAuth redirect handler
/api/tutor/stream	POST	Session cookie	SSE AI tutor token stream
/api/stripe/webhook	POST	Stripe signature	Stripe event processing
/manus-storage/*	GET	None (signed URLs)	S3 file proxy

10. Payment & Subscription Architecture

Stripe Products & Prices

Four Stripe products were created via `scripts/create-stripe-products.mjs` in the sandbox Stripe account. The Price IDs are hardcoded in `server/stripe.ts` and must be updated when switching to live mode.

Plan	Billing	Amount	Stripe Price ID
Family Plan	Monthly	\$19.99/mo	price_1TbnYY7McfD3gqtzhIiuU8AG
Family Plan	Annual	191.88/ <i>yr</i> (15.99/mo)	price_1TbnYa7McfD3gqtz6r8qjbHx
Premium Family Plan	Monthly	\$29.99/mo	price_1TbnYd7McfD3gqtzJc4cBA06
Premium Family Plan	Annual	287.88/ <i>yr</i> (23.99/mo)	price_1TbnYf7McfD3gqtzcQzNuisb

Checkout Flow

1. User selects a plan and billing period on the landing page pricing section.
2. If the user is not logged in, `RoleSelectModal` opens to prompt sign-in. The selected plan and billing period are stored in `sessionStorage` as `selectedPlan` and `billingPeriod`.
3. If the user is already logged in, `CheckoutModal` opens directly. The user can enter a coupon code; the `payment.validateCoupon` procedure validates it in real time and shows the adjusted price.
4. The user clicks "Proceed to Checkout". The frontend calls `payment.createCheckoutSession`, which creates a Stripe Checkout Session with `mode: "subscription"`, the correct `price ID`, `allow_promotion_codes: true`, and metadata containing `user_id`, `plan_key`, `billing_period`, and `coupon_id`.
5. The server returns the Stripe Checkout URL. The frontend opens it in a new tab via `window.open(url, '_blank')`.
6. After payment, Stripe redirects to `/checkout/success?session_id={CHECKOUT_SESSION_ID}`.
7. Stripe fires `checkout.session.completed` to the webhook endpoint, which upserts the subscription, saves the billing period, and records the coupon redemption.

Webhook Events Handled

Event	Action
<code>checkout.session.completed</code>	Upsert subscription, save billing period, record coupon redemption, log payment event
<code>customer.subscription.created</code>	Upsert subscription with full period details
<code>customer.subscription.updated</code>	Update subscription status, period, cancel flag
<code>customer.subscription.deleted</code>	Mark subscription as canceled
<code>invoice.paid</code>	Log payment event
<code>invoice.payment_failed</code>	Set subscription status to <code>past_due</code> , log event

Webhook Security

The webhook handler at `/api/stripe/webhook` uses `express.raw({ type: 'application/json' })` registered **before** `express.json()` in `server/_core/index.ts`. This is critical — if `express.json()` parses the body first, `stripe.webhooks.constructEvent()` will fail because it needs the raw buffer for HMAC verification.

Test events (where `event.id` starts with `evt_test_`) are detected before signature verification and immediately return `{ verified: true }` to satisfy the Manus Stripe integration test.

If `STRIPE_WEBHOOK_SECRET` is not set, the server logs a warning and processes the event without signature verification. This is acceptable in development but must be resolved before production traffic.

Billing Portal

Users can access the Stripe Customer Portal (to update payment methods, cancel, or download invoices) via the `/billing` page, which calls `payment.getBillingPortalUrl`. The portal URL is generated server-side using `stripe.billingPortal.sessions.create()`.

11. Coupon & Promotional Discount System

Coupon Data Model

Coupons are stored in the `coupons` table with the following configurable parameters:

- **Discount type:** `percentage` (e.g. 20%) or `fixed` (e.g. \$5.00 in cents).
- **Applicability:** `applicablePlans` (JSON array of plan keys; null = all plans), `eligibility` (all/new_users/parents/students/schools/selected), `selectedUserIds` (for targeted coupons), `minAmount` (minimum subscription price in cents).
- **Usage limits:** `usageLimit` (global; null = unlimited), `perUserLimit` (default 1), `usageCount` (incremented on each redemption).
- **Duration:** `once` (first billing cycle only), `repeating` (for `durationMonths` cycles), or `forever`.
- **Validity:** `startDate`, `expiresAt`, `status` (active/paused/expired/archived).
- **Stacking:** `isStackable` (default false).
- **Stripe sync:** `stripeCouponId` and `stripePromotionCodeId` are set when the admin creates a coupon and the Stripe coupon is mirrored.

Validation Logic (`payment.validateCoupon`)

The `validateCoupon` procedure enforces the following rules in order:

1. Coupon code exists and status is `active`.

2. `startDate` has passed (if set).
3. `expiresAt` has not passed (if set).
4. `usageLimit` has not been reached (if set).
5. Per-user limit: the user has not already redeemed this coupon more than `perUserLimit` times.
6. Plan applicability: the requested plan is in `applicablePlans` (or the list is empty/null).
7. Minimum amount: the plan price meets `minAmount` (if set).
8. Eligibility: `new_users` checks that the user has no prior subscription; `selected` checks `selectedUserIds`.

If all checks pass, `calculateDiscount()` in `server/stripe.ts` computes the final price and returns a typed `CouponValidationResult`.

Admin Coupon Management

Admins create and manage coupons from the **Coupon Manager** tab in the Admin Console. Creating a coupon optionally mirrors it to Stripe via `stripe.coupons.create()` and `stripe.promotionCodes.create()`, so it can also be applied directly in the Stripe Checkout UI via the `allow_promotion_codes: true` flag.

12. Parent/Student Onboarding Workflows

Student Onboarding (3 steps)

Step 1 — Profile: The student provides date of birth, grade level, school type, school district, and school name. Age is derived from date of birth; implausible age/grade combinations are blocked client-side. The profile is saved via `onboarding.saveStudentProfile`.

Step 2 — Invite Parent (optional): The student can invite a parent by email or phone. The system checks whether the email already has a Manus account. If so, a parent invite token is created and a branded email is sent. If not, the invite email includes a sign-up link. The student can also accept a pending parent invite token from the URL (`?parentInvite=TOKEN`).

Step 3 — Complete: `onboarding.completeOnboarding` marks the profile complete and auto-enrolls the student in the grade-appropriate default course if they have no existing enrollment.

Parent Onboarding (3 steps)

Step 1 — Profile: The parent provides demographic information and a free-text signup reason, plus a goal category (grade improvement, test prep, enrichment, remediation, homeschool supplement, or other).

Step 2 — AI Goal Alignment: The `onboarding.generateGoalAlignment` procedure calls the Manus Forge LLM with a structured prompt that incorporates the parent's signup reason, goal category, child's name, grade, and school type. The LLM returns a 3–4 sentence personalised goal statement that is displayed to the parent and saved to their profile as `parentGoalDetail`.

Step 3 — Complete: Same as student flow. Parents are not auto-enrolled in courses (they are not learners), but the `completeOnboarding` mutation marks their profile as complete.

Plan/Billing Context Persistence

When a user clicks a pricing CTA, the selected plan key and billing period are written to `sessionStorage` as `selectedPlan` and `billingPeriod`. These values persist through the OAuth redirect (`sessionStorage` survives page navigation within the same tab). On onboarding completion, the `payment.saveBillingPeriod` procedure reads the value and persists it to `users.billingPeriod`.

Invite Token System

Two invite token flows exist:

Parent → Student: A parent generates a student invite token (stored in `studentInviteTokens`). The token is embedded in a URL shared with the child. When the child signs up or logs in with that URL, `onboarding.acceptStudentInvite` creates a `parentChildLinks` record.

Student → Parent: A student generates a parent invite token (stored in `parentInviteTokens`). A branded email is sent to the parent's email address. The parent clicks the link, which calls `onboarding.acceptParentInvite`, creating the `parentChildLinks` record and optionally upgrading the parent's account type.

13. AI Tutor Architecture (EduBot)

EduBot is the platform's AI learning coach. It is a streaming, context-aware LLM assistant that adapts its behaviour based on the student's mastery data, placement test results, course context, and the active interaction mode.

Streaming Endpoint

EduBot does not use tRPC for its primary interaction. Instead, it uses a dedicated **Server-Sent Events (SSE)** endpoint at `POST /api/tutor/stream`. This design was chosen because tRPC does not natively support streaming token-by-token responses.

The request body carries: `{ message, unitNumber, mode, sessionId }`. The server authenticates the request via the session cookie, builds the full system prompt, and streams the LLM response token-

by-token back to the client using the SSE protocol (`data: {"type":"token","content":"..."}` lines). A final `data: {"type":"done"}` event signals completion.

The model used is **Gemini 2.5 Flash** via the Manus Forge API (`BUILT_IN_FORGE_API_URL`). The request uses `stream: true` and `max_tokens: 4096` .

Tutor Modes

Six interaction modes are defined in `server/educhamp-helpers.ts` . Each mode produces a different system prompt section that governs the tutor's behaviour:

Mode	Purpose	Key Behaviour
<code>teach</code>	Explain concepts	2–4 paragraph explanations, checking questions, real-world examples
<code>practice</code>	Guided problem-solving	Difficulty adapts to mastery level; one problem at a time; hints before solutions
<code>quiz</code>	Formative assessment	One question at a time; no hints before answer; performance summary at end
<code>exam_review</code>	Test preparation	Prioritises units below 75% mastery; builds personalised review checklist
<code>remediation</code>	Targeted gap filling	Starts from the most basic level; multiple representations; celebrates small wins
<code>parent_summary</code>	Parent progress report	Plain-language report with 6 structured sections; uses actual mastery data

System Prompt Construction

The `buildTutorSystemPrompt()` function in `server/educhamp-helpers.ts` assembles a multi-section system prompt that is injected before every conversation. The prompt includes:

- Identity block** — EduBot's name, subject specialisation, personality, and first-message introduction template.
- Student information** — preferred name, course, current unit, grade level, school type, and school district.
- Mode instructions** — the full mode-specific behavioural guidelines from `MODE_INSTRUCTIONS` .
- Placement test results** — overall diagnostic score, placement recommendation, and per-unit readiness breakdown.
- Mastery data table** — all tracked skills categorised by level (Beginner/Developing/Approaching/Mastered/Advanced) with skill IDs and scores.

- 6. **Unit-by-unit mastery overview** — average mastery, quiz score, and status for every unit in the course (used in `parent_summary` mode).
- 7. **Recent quiz performance** — last 5 quiz scores with dates.
- 8. **Current unit learning objectives** — TEKS-aligned objectives from the lesson records.
- 9. **Adaptive pacing guidance** — average mastery score, adaptive path label, and specific instruction style recommendation.
- 10. **Parent goal alignment context** — goal category, AI-generated goal statement, and raw signup reason (injected when a parent views a child's session).
- 11. **Out-of-course redirection** — explicit instruction to redirect questions about other subjects back to the active course.
- 12. **Core principles** — 8 behavioural rules including TEKS alignment, personalisation, and tone.

Mastery Levels

Mastery scores (0–100) are classified into five levels by `getMasteryLevel()` :

Level	Score Range	Adaptive Path
Beginner	< 60	Reteach
Developing	60–74	Guided practice
Approaching	75–89	Quiz unlocked
Mastered	90–99	Challenge
Advanced	100	Challenge

Session Persistence

Tutor conversation history is stored in the `tutorSessions` table as a JSON array of `{ role, content }` messages. The `tutor.getSession` procedure retrieves or creates a session for a given `(userId, unitId, mode)` combination. The `tutor.updateSession` procedure appends messages after each exchange. The streaming endpoint reads the last 20 messages from the session to build the conversation history injected into the LLM request.

14. Curriculum & Course Structure

Course Catalogue

The `courses` table stores all available courses. Each course has a `gradeLevel` (e.g. “3”, “9”, “AP”), a `subject` (math, science, english, social_studies, language, business, test_prep, other), a `courseCode`

(e.g. “ALG1”, “APCHEM”), and an optional `teksCode` (e.g. “TEKS 111.39”). The `isDefault` flag marks the grade-appropriate course that new students are auto-enrolled in when they complete onboarding.

The platform currently offers **56+ courses** spanning Grades 3–12 and AP levels, all TEKS-aligned.

Curriculum Hierarchy

```
Course
├─ Unit (unitNumber, sortOrder)
│   ├── Lesson (sortOrder, content JSON, teksAlignment)
│   └─ QuizQuestion (difficulty, skillTag, questionType)
└─ Skill (skillId format: {courseCode}-U{N}-S{N})
```

Each unit has a `unitNumber` (1-based integer used throughout the system for ordering and skill ID construction) and a `sortOrder` for display. Units contain lessons and quiz questions. Skills are TEKS-aligned competencies associated with a unit and course; their IDs follow the pattern `{courseCode}-U{unitNumber}-S{skillNumber}` (e.g. ALG1-U3-S2).

Unit Progress State Machine

A student's progress through a unit follows a strict state machine stored in `unitProgress.status`:

```
locked → in_progress → quiz_unlocked → completed
```

- **locked** — the unit is not yet accessible (prerequisite units not completed).
- **in_progress** — the student has started at least one lesson in the unit.
- **quiz_unlocked** — all lessons in the unit have been marked complete; the quiz is now accessible.
- **completed** — the student has passed the unit quiz (score $\geq 70\%$).

The `progress.markLessonComplete` procedure checks whether all lessons in the unit are now complete and, if so, transitions the unit status to `quiz_unlocked`.

Diagnostic Placement Test

The diagnostic test (`diagnosticQuestions` table) is a per-course assessment that covers representative questions from every unit. When a student submits a diagnostic attempt via `diagnostic.submit`:

1. Each answer is scored and mapped to a `skillTag` and `unitNumber`.
2. Per-unit scores are computed and stored in `diagnosticAttempts.unitResults` (JSON).
3. An overall score and placement recommendation are generated.

4. All units where the student scored $\geq 75\%$ are marked as “ready to skip” in the placement results.
5. The diagnostic results are injected into the EduBot system prompt for all subsequent tutor sessions.

A `diagnosticCooldownDays` field on the `courses` table prevents students from retaking the diagnostic too frequently.

Quiz Engine

The `quiz.getQuestions` procedure returns questions for a unit without revealing correct answers. The `quiz.submitQuiz` procedure:

1. Grades each answer and computes a score (0–100).
 2. Updates `unitProgress.quizScore` and `unitProgress.quizAttempts`.
 3. If the score is $\geq 70\%$, transitions the unit status to `completed`.
 4. Updates per-skill mastery scores in `userMastery` using a weighted average: $\text{newScore} = (\text{oldScore} \times 0.7) + (\text{questionScore} \times 0.3)$.
 5. Records the full attempt in `quizAttempts`.
-

15. Admin Portal Capabilities

The Admin Console is a tabbed interface at `/admin` accessible only to users with `role = 'admin'`. It is built as a single `AdminDashboard.tsx` page that renders tab-specific components.

Admin Console Tabs

Tab	Component	Key Capabilities
Overview	Inline in AdminDashboard	Platform KPIs: total users, students, parents, courses, enrollments, quiz attempts, diagnostic attempts, active subscriptions, MRR
Users	Inline	Search, filter by status/type, role promotion, account type change, manual course enrollment, user deletion
Courses	Inline	Course list, create/edit course metadata, publish/unpublish, set default course per grade
Demo Requests	DemoRequestsTab	CRM table with status pipeline, assignment, admin notes, branded email reply, stats cards
Coupons	CouponManagerTab	Create/edit/archive coupons, usage stats, Stripe coupon mirroring
Subscriptions	SubscriptionCRMTab	Paginated subscription list with status filter, plan/billing period, renewal date
Payment Analytics	PaymentAnalyticsTab	MRR, active subscriptions, coupon redemptions, failed payments, plan distribution chart, billing period split chart, recent events, CSV export
Newsletter	Inline	Subscriber list, create campaign, AI-drafted campaign via LLM, send/mark sent
RBAC	Inline	Create/edit roles, assign permissions, assign roles to users
CMS	Inline	Draft/publish/revert content for lessons, units, and courses; version history
Audit Log	Inline	Immutable log of all admin actions with user, action, resource, and IP
Settings	Inline	Platform-wide key-value configuration store
Email Logs	Inline	Outbound email audit trail with status, template name, and error messages

Admin Promotion

To promote a user to admin, update the `role` column directly in the database:

```
UPDATE users SET role = 'admin' WHERE email = 'admin@example.com';
```

This can be done via the Database panel in the Management UI or via `webdev_execute_sql`.

16. Notification Systems

Transactional Email (Resend)

All transactional emails are sent via the **Resend** API using the helper in `server/emailService.ts`. The default sender address is `EduChamp <invites@educhamp.app>`. The service retries failed sends up to 3 times with exponential backoff. Every send attempt (successful or failed) is logged to the `emailLogs` table.

Critical: The domain `educhamp.app` must be verified in the Resend dashboard with SPF, DKIM, and DMARC DNS records before transactional emails will be delivered. See Section 21 for the required DNS records.

Branded email templates are defined in `server/emailTemplates/`:

Template	Trigger	Recipient
<code>parentInvite.ts</code>	Student invites parent	Parent's email
<code>demoRequestConfirmation.ts</code>	School submits demo request	Requester's email
<code>demoRequestAdminNotification.ts</code>	School submits demo request	Admin team
<code>demoRequestResponse.ts</code>	Admin responds to demo request	Requester's email

In-App Notifications

The `userNotifications` table stores per-user notification records. The scheduled `inviteExpiry.ts` heartbeat creates in-app notifications when parent invite tokens expire. The frontend reads these notifications via a tRPC procedure and displays them in the dashboard notification feed.

Owner Alerts (Manus Notification Service)

The `notifyOwner()` helper in `server/_core/notification.ts` sends push notifications to the Manus project owner via the Manus Forge notification API. It is used for operational alerts such as new demo request submissions and newsletter campaign completions.

SMS / Phone Notifications

SMS follow-up for demo requests is captured as a field in the `demoRequests` table (`phone`). No SMS provider is currently integrated. The recommended integration path is **Twilio** via a new tRPC procedure in the admin router that calls the Twilio REST API when an admin triggers a follow-up.

17. Landing Page & Conversion Funnel

The landing page (`client/src/pages/LandingPage.tsx`) is the primary public-facing conversion surface. It contains the following sections in order:

1. **Top navigation** — logo, nav links (Features, Curriculum, Schools, Pricing, FAQ), Sign In / Get Started CTAs, mobile hamburger menu.
2. **Hero section** — headline, sub-headline, primary CTA (“Start Free Trial”), trust badges.
3. **Features section** — three-column feature grid.
4. **How It Works section** — step-by-step process.
5. **Schools & Districts section** (`id="schools"`) — ISD-targeted hero with four benefit pillars, district badge strip, stats row, and “Request a Demo for Your District” CTA wired to `RequestDemoModal`.
6. **Pricing section** — monthly/annual billing toggle (saves 20%), three plan cards (Family, Premium Family, ISD/School), plan comparison table (16 feature rows).
7. **FAQ section** — accordion.
8. **Footer** — nav links, newsletter signup, social links.

Anonymous Landing Chatbot

An embedded `LandingChatbot` component (lines 26–260 of `LandingPage.tsx`) provides an anonymous AI assistant for pre-signup visitors. It creates an anonymous session via `landing.createSession`, sends messages via `landing.chat` (which calls the LLM with a landing-specific system prompt that avoids quoting prices), and after approximately 3 messages prompts the visitor to provide their name and email for follow-up. Contact info is saved via `landing.updateSessionContact`.

Pricing CTA Behaviour

- **Not logged in:** clicking any paid plan CTA stores the plan key and billing period in `sessionStorage`, then opens `RoleSelectModal` to prompt sign-in. After OAuth, the user is redirected back and `CheckoutModal` opens automatically.
 - **Logged in:** `CheckoutModal` opens directly with the selected plan pre-filled.
 - **ISD/School plan:** opens `RequestDemoModal` regardless of login state.
-
-

18. Security Architecture

Security Headers (Helmet)

Helmet is mounted as the first middleware in `server/_core/index.ts`. In production, the full Helmet default CSP is active. In development, CSP is disabled to allow Vite HMR inline scripts. `crossOriginEmbedderPolicy` is disabled globally to allow the Manus storage proxy to serve assets cross-origin.

Rate Limiting

Two `express-rate-limit` limiters are applied:

Limiter	Path	Limit	Window
<code>apiLimiter</code>	<code>/api/trpc</code>	300 req/min per IP	60 seconds
<code>chatbotLimiter</code>	<code>/api/tutor/stream</code>	20 req/5 min per IP	5 minutes

OAuth callback paths (`/api/oauth/*`) are explicitly excluded from rate limiting to prevent lockouts during authentication.

`trust proxy: 1` is set so Cloud Run's `X-Forwarded-For` header is used for real client IP resolution.

Authentication & Session Management

Authentication is handled entirely by **Manus OAuth**. There are no passwords stored in the application database. The session lifecycle is:

1. User clicks "Sign In" → frontend calls `getLoginUrl()` which encodes `window.location.origin` and a `returnPath` in the OAuth state parameter.
2. Manus OAuth redirects to `/api/oauth/callback` with an authorization code.
3. The server exchanges the code for a Manus access token, looks up or creates the local user record, and issues a **JWT session cookie** (signed with `JWT_SECRET`, `httpOnly`, `sameSite: lax`).
4. Every subsequent request to `/api/trpc` calls `verifySession()` in `server/_core/sdk.ts` to decode the JWT and load the user from the local DB.
5. Users with `status = 'suspended'` or `status = 'deleted'` are rejected at the `authenticateRequest` layer.

Known gap: Users with `status = 'archived'` are not currently blocked at the `authenticateRequest` layer. This should be added before production launch.

RBAC (Role-Based Access Control)

Three built-in roles exist: `admin`, `user`, and `student`. Custom roles can be created in the Admin Console with per-resource, per-action permissions. The `hasPermission` middleware in `server/routers.ts` gates admin procedures by checking the user's effective permissions (union of all assigned roles). The `adminProcedure` guard requires `role === 'admin'` at the user table level, independent of the RBAC system.

Payment Security

- Raw card data is never stored in the application database. All payment method handling is delegated to Stripe.
- Stripe webhook signatures are verified using `stripe.webhooks.constructEvent()` with the `STRIPE_WEBHOOK_SECRET`. Test events (IDs starting with `evt_test_`) are short-circuited with a `{ verified: true }` response.
- The Stripe webhook route is registered **before** `express.json()` to preserve the raw request body required for signature verification.

TOTP / Two-Factor Authentication

The `twoFactorAuth` table stores TOTP secrets (encrypted at rest by the DB layer) and 8 one-time backup codes. The `auth.setup2FA`, `auth.verify2FA`, `auth.disable2FA`, and `auth.generateBackupCodes` procedures implement the full 2FA lifecycle using the `speakeasy` library.

19. Deployment Architecture

Runtime Environment

The application runs as a **single Node.js process** on **Google Cloud Run** via the Manus managed hosting platform. Key constraints:

- **No persistent filesystem** — all file storage must use S3 (via `storagePut` / `storageGet` helpers).
- **No background workers** — `setInterval`, `node-cron`, and long-lived processes are not supported. All scheduled work must use the Heartbeat cron pattern.
- **1 vCPU, 512 MiB RAM** — avoid per-request heavy compute.
- **180-second request timeout** — long-running operations must be broken into smaller steps.
- **Min instances = 0** — cold starts will occur after periods of inactivity.

Build & Serve Strategy

In development, Vite runs as middleware within the Express server (`setupVite(app, server)`). In production, the Vite build output is served as static files by Express (`serveStatic(app)`). The frontend and backend share a single port.

Scheduled Jobs (Heartbeat Pattern)

Scheduled jobs use the Manus Heartbeat cron system. Jobs are registered as HTTP POST endpoints in `server/_core/index.ts` and scheduled via the `manus-config schedule` CLI. The platform calls the registered endpoint on the configured schedule.

Job	Endpoint	Schedule	Task UID
Grade Promotion	<code>/api/scheduled/grade-promotion</code>	Annually (end of year)	Configured in <code>server/scheduledHandlers.ts</code>
Invite Expiry Scan	<code>/api/scheduled/invite-expiry</code>	Daily 02:00 UTC	<code>R3xaGZyn92oQu2Wwj8FTeP</code>

Critical rule: All heartbeat handlers must be idempotent. They authenticate requests using `sdk.authenticateRequest` with the `cron_` identity prefix.

Environment Variables

All environment variables are injected by the Manus platform at runtime. They must never be committed to the repository.

Variable	Purpose	Source
DATABASE_URL	MySQL/TiDB connection string	Platform-injected
JWT_SECRET	Session cookie signing key	Platform-injected
VITE_APP_ID	Manus OAuth application ID	Platform-injected
OAUTH_SERVER_URL	Manus OAuth backend base URL	Platform-injected
VITE_OAUTH_PORTAL_URL	Manus login portal URL (frontend)	Platform-injected
OWNER_OPEN_ID	Owner's Manus Open ID	Platform-injected
OWNER_NAME	Owner's display name	Platform-injected
BUILT_IN_FORGE_API_URL	Manus Forge API base URL (LLM, storage, notifications)	Platform-injected
BUILT_IN_FORGE_API_KEY	Forge API bearer token (server-side)	Platform-injected
VITE_FRONTEND_FORGE_API_KEY	Forge API bearer token (frontend)	Platform-injected
VITE_FRONTEND_FORGE_API_URL	Forge API URL (frontend)	Platform-injected
RESEND_API_KEY	Resend transactional email API key	Settings → Secrets
RESEND_FROM_EMAIL	Sender address (default: EduChamp <invites@educhamp.app>)	Settings → Secrets
STRIPE_SECRET_KEY	Stripe secret key	Settings → Payment
VITE_STRIPE_PUBLISHABLE_KEY	Stripe publishable key (frontend)	Settings → Payment
STRIPE_WEBHOOK_SECRET	Stripe webhook signing secret	Settings → Payment
VITE_APP_TITLE	App display name in browser tab	Settings → General
VITE_APP_LOGO	App logo URL	Settings → General

20. Testing Strategy

Test Runner

Vitest is used as the test runner. Test files are located in `server/**/*.test.ts` and `server/**/*.spec.ts`. The test environment is Node (not browser). Path aliases (`@`, `@shared`) are configured in `vitest.config.ts`.

Test Count (as of Sprint 26)

141 tests across 6 test files, all passing. TypeScript: 0 errors.

File	Tests	Coverage Area
<code>server/auth.logout.test.ts</code>	~8	Auth logout, session invalidation
<code>server/educhamp.test.ts</code>	26	Mastery logic, adaptive path, skill ID format, tutor prompts
<code>server/authEnhancements.test.ts</code>	30	TOTP/2FA, password reset tokens, parent-child access, skill gap analysis, CSV export
<code>server/demoRequest.test.ts</code>	33	Demo request schema validation, enum constraints, pagination, interest label mapping
<code>server/payment.test.ts</code>	36	calculateDiscount, getPlanByKey, coupon validation (expiry/limits/plan/eligibility), billing period, subscription upsert, payment event logging, admin CRUD
<code>server/inviteExpiry.test.ts</code>	~8	Heartbeat expiry logic, notification creation

Test Style

Tests are primarily **unit-style with mocked dependencies**. The `./db` and `./stripe` modules are mocked in payment tests. Business logic functions (`calculateDiscount`, `getMasteryLevel`, `buildTutorSystemPrompt`, etc.) are tested directly without HTTP layer involvement.

There are no end-to-end browser tests. Integration testing is done manually via the dev server preview.

Running Tests

```
cd /home/ubuntu/educhamp
pnpm test           # Run all tests once
pnpm test --watch   # Watch mode
npx tsc --noEmit    # TypeScript type check (no output)
```

21. Known Issues & Technical Debt

The following items are documented as known gaps or deferred work:

Issue	Severity	Status	Notes
Resend DNS not verified for <code>educhamp.app</code>	P0	Blocked (manual step)	SPF, DKIM, DMARC records must be added to DNS provider; see Section 22
Stripe webhook secret not registered	P0	Blocked (manual step)	Requires Stripe dashboard → Developers → Webhooks; copy <code>whsec_</code> secret to Settings → Payment
Stripe Price IDs are sandbox-only	P0	Blocked (manual step)	Live Price IDs must be created after Stripe KYC; update <code>PLANS</code> in <code>server/stripe.ts</code>
<code>archived</code> users not blocked at auth layer	P1	Open	<code>authenticateRequest</code> in <code>sdk.ts</code> blocks <code>suspended</code> and <code>deleted</code> but not <code>archived</code>
District logos on Schools section are text badges	P2	Blocked	Awaiting signed partnership agreements with Katy ISD, Spring ISD, Cy-Fair ISD, Humble ISD, Conroe ISD, Alief ISD
No end-to-end browser tests	P2	Open	All integration testing is manual; Playwright or Cypress suite recommended
Tutor session history capped at 20 turns	P3	By design	Prevents unbounded DB growth; increase cap in <code>tutorStream.ts</code> if needed
No client-side error monitoring (Sentry)	P3	Open	<code>ErrorBoundary</code> shows user-friendly message but does not report to any monitoring service
SMS/phone follow-up for demo requests	P3	Open	Phone field captured but no Twilio integration; add <code>server/smsService.ts</code> when ready
PayPal/ACH in Stripe may require additional Stripe account configuration	P3	Open	PayPal and ACH are enabled in the checkout session <code>payment_method_types</code> but may require activation in the Stripe dashboard under Settings → Payment Methods
Annual billing period not yet passed to Stripe subscription metadata	P3	Open	<code>billingPeriod</code> is stored in the <code>users</code> table and <code>subscriptions</code> table but the Stripe Price ID already encodes the interval; no action needed unless custom metadata is required

22. Resend DNS Verification Guide

To enable transactional email delivery from `invites@educamp.app`, the following DNS records must be added to the domain registrar or DNS provider for `educamp.app`:

1. Log in to resend.com/domains and click **Add Domain**.
2. Enter `educamp.app` (or a subdomain such as `mail.educamp.app`).
3. Resend will display the exact record values. Add the following record types to your DNS provider:

Type	Name	Value
TXT	<code>resend._domainkey</code>	DKIM public key (provided by Resend)
TXT	<code>@</code> or <code>educamp.app</code>	<code>v=spf1 include:amazonses.com ~all</code>
TXT	<code>_dmarc</code>	<code>v=DMARC1; p=none; rua=mailto:dmarc@educamp.app;</code>

1. Click **Verify DNS Records** in the Resend dashboard. Verification typically completes within minutes on Cloudflare; other providers may take up to 48 hours.
2. Once verified, upgrade DMARC policy from `p=none` to `p=quarantine` after confirming all emails pass SPF/DKIM alignment.

Cloudflare shortcut: Resend supports one-click automatic DNS setup for Cloudflare-managed domains. Authorise Resend to access your Cloudflare account and all records are added automatically.

23. Roadmap & Future Enhancements

The following features are planned but not yet implemented:

Near-term (Sprint 27–30):

- **Subscription upgrade/downgrade flow** — “Change Plan” button on the `/billing` page that opens the Stripe Customer Portal or a dedicated upgrade modal.
- **Free trial period** — add `subscription_data: { trial_period_days: 14 }` to the Stripe checkout session for new subscribers.
- **Twilio SMS integration** — send SMS follow-up notifications to demo request submitters.
- **Sentry error monitoring** — integrate Sentry for client-side and server-side error reporting.
- **Playwright E2E tests** — automated browser tests for critical flows (sign-up, onboarding, diagnostic, quiz, checkout).

Medium-term (Sprint 31–40):

- **ISD/School district licensing** — multi-seat billing, district admin portal, bulk student import via CSV, SSO integration (Google Workspace, Clever).
- **Teacher role** — `accountType: 'teacher'` with class management, assignment creation, and student progress monitoring.
- **Live tutoring sessions** — video/audio integration for scheduled 1:1 sessions with human tutors.
- **Mobile app** — React Native wrapper for iOS and Android using the existing tRPC API.
- **Gamification** — XP points, badges, leaderboards, and streak tracking.

Long-term:

- **State-wide TEKS alignment expansion** — extend beyond Texas to other state standards (Common Core, NGSS).
- **Content authoring tools** — allow teachers and district curriculum coordinators to create custom courses and lessons.
- **AI-generated practice problems** — use the LLM to generate unlimited unique practice problems at the correct difficulty level for each student.

24. UI/UX Design Standards

Design System

The platform uses **shadcn/ui** components built on **Radix UI** primitives with **Tailwind CSS 4** for styling. All component imports use the `@/components/ui/*` path alias.

Color Palette

The active theme is **dark** (dark backgrounds, light text). CSS variables are defined in `client/src/index.css` under the `.dark {}` selector. Six personalisation palettes are available: indigo (default), emerald, rose, violet, amber, and teal. Students can select their palette on the Profile page.

Typography

- **Body font:** Inter (Google Fonts CDN)
- **Display/heading font:** Lora (Google Fonts CDN)
- Both fonts are loaded via `<link>` tags in `client/index.html`.

Animation Standards

- UI animations are kept under 300ms. Buttons use `transform: scale(0.97)` on `:active`.
- Modals/drawers use 200–350ms ease-out transitions.

- Only `transform` and `opacity` are animated (GPU-accelerated).
- `prefers-reduced-motion` is respected via `@media` queries.

Reusable Components

Component	Path	Purpose
<code>DashboardLayout</code>	<code>client/src/components/DashboardLayout.tsx</code>	Authenticated app shell with collapsible sidebar
<code>AIChatBox</code>	<code>client/src/components/AIChatBox.tsx</code>	Full-featured chat interface with streaming support
<code>CheckoutModal</code>	<code>client/src/components/CheckoutModal.tsx</code>	Stripe checkout with coupon entry and price preview
<code>RequestDemoModal</code>	<code>client/src/components/RequestDemoModal.tsx</code>	3-step ISD/school lead capture form
<code>RoleSelectModal</code>	<code>client/src/components/RoleSelectModal.tsx</code>	Parent/Student role selection before sign-up
<code>CourseSwitcher</code>	<code>client/src/components/CourseSwitcher.tsx</code>	Browse and switch between enrolled courses
<code>ErrorBoundary</code>	<code>client/src/components/ErrorBoundary.tsx</code>	Client-side error fallback with dev stack trace

25. Naming Conventions & Coding Standards

File Naming

- React components: `PascalCase.tsx` (e.g. `DashboardLayout.tsx`)
- Server files: `camelCase.ts` (e.g. `emailService.ts`, `stripeWebhook.ts`)
- Router files: `server/routers/{feature}.ts` (e.g. `payment.ts`, `onboarding.ts`)
- Test files: `server/{feature}.test.ts`
- Migration scripts: `scripts/{purpose}.mjs` (ES modules, `.mjs` extension)

tRPC Procedure Naming

Procedures follow the pattern `{router}.{action}{Resource}`:

- Queries: `getX`, `listX`, `searchX`
- Mutations: `createX`, `updateX`, `deleteX`, `archiveX`, `submitX`

- Admin procedures are nested under `admin.X` or `payment.admin.X`

Database Column Naming

All columns use `camelCase` in Drizzle schema definitions. The underlying MySQL columns use the same names. Timestamps are stored as `bigint` (milliseconds since epoch).

Skill ID Format

Skills follow the pattern `{courseCode}-U{unitNumber}-S{skillNumber}` (e.g. `ALG1-U3-S2`). This format is enforced in the tutor system prompt and referenced throughout the mastery tracking system.

26. Developer Setup Instructions

Prerequisites

- Node.js 22.x
- pnpm 9.x
- Access to the Manus platform (for environment variables and database)

Local Development

```
# Clone the repository
git clone <repository-url>
cd educchamp

# Install dependencies
pnpm install

# Start the development server (Express + Vite on port 3000)
pnpm dev

# Run tests
pnpm test

# TypeScript type check
npx tsc --noEmit

# Generate a new Drizzle migration after schema changes
pnpm drizzle-kit generate

# Apply a migration (read the generated .sql file, then run via webdev_execute_sql)
# Never run drizzle-kit push directly – use webdev_execute_sql to apply migrations
```


Schema Migration Workflow

1. Edit `drizzle/schema.ts`.
2. Run `pnpm drizzle-kit generate` to produce a new `.sql` file in `drizzle/`.
3. Read the generated `.sql` file to review the changes.
4. Apply via `webdev_execute_sql` (Manus Management UI → Database, or the `webdev_execute_sql` tool).
5. Update `drizzle/meta/_journal.json` if needed to track the migration as applied.

Adding a New tRPC Procedure

1. Add a query helper in `server/db.ts` if a new DB query is needed.
 2. Add the procedure in the appropriate router file under `server/routers/`.
 3. Import and register the router in `server/routers.ts` if it is a new router.
 4. Consume the procedure in the frontend via `trpc.{router}.{procedure}.useQuery()` or `.useMutation()`.
 5. Write a Vitest test in `server/{feature}.test.ts`.
-

27. Change Log & Major Milestones

Sprint	Key Deliverables
Phase 1–2	Project scaffold, DB schema, Algebra I curriculum seed (12 units, 57 diagnostic questions, 100+ quiz questions)
Phase 3–4	Dashboard, curriculum browser, lesson viewer, guided/independent practice
Phase 5	AI Tutor (EduBot) with 6 modes, SSE streaming, session persistence
Phase 6	Diagnostic placement test, scoring, placement recommendations
Phase 7	Quiz engine, mastery scoring, adaptive path unlock logic
Phase 8	Mastery & progress dashboard, skill index, recharts visualisations
Phase 9	Guardian notifications, 26 Vitest tests, loading skeletons, micro-interactions
Round 2	Tutor UX improvements, context-aware prompts, Parent Summary mode
Parent Module	Multi-child management, parent dashboard, co-parent invitation system
Auth Enhancements	TOTP/2FA, password reset, parent goals/notes, skill gap analysis, report export
Referral System	Referral codes, student invite tokens, parent-only registration, onboarding wizards
Sprint 6	Variable diagnostic retest, expanded question banks (57 questions/course)
Sprint 7	Admin module, multi-course expansion (9th Grade English I, Biology I, AP Human Geography, Spanish 2, Gr 3 subjects)
Sprint 8	Grade-filtered courses, progression locks, answer normalisation, UX fixes
Sprint 9	Multi-course enrollment, CourseSwitcher, guided first-login tour
Sprint 10	AP/SAT courses (AP Chemistry, AP Statistics, AP Calculus BC, AP Literature, AP Business, SAT Prep)
Sprint 11	Course-aware diagnostic, personalisation palettes, preferred name
Sprint 12	CourseWelcome gate, diagnostic question bank expansion to all courses
Sprint 13	Role-based tutor, landing page redesign, onboarding improvements
Sprint 14	Newsletter console, AI News Bot, landing chatbot lead capture, admin chat management
Sprint 15	EduBot personality, course guardrails, Katy ISD Gr 4–8 catalogue (ACA + KAP)
Sprint 16	AP diagnostic fix, grade-aware auto-enrolment, expanded diagnostic banks
Sprint 17	FAQ/landing updates, onboarding fixes, intelligent course enrollment, QA/UAT (68 tests)

Sprint	Key Deliverables
Sprint 18	Diagnostic course name fix, EduChamp logo/favicon, mobile responsiveness
Sprint 19	Course-specific diagnostic cooldown, enhanced user/course management, CMS, RBAC
Sprint 20	Student-to-parent invitation workflow redesign, branded email templates
Sprint 21	Transactional email (Resend), invite status banner, resend workflow
Sprint 22	Custom Resend sender domain, Email Logs admin tab, invite expiry heartbeat
Sprint 23	ISD demo request system (CRM), pricing enhancements (annual toggle, comparison table), 105 tests
Sprint 24	DNS verification guide, Schools & Districts hero section, annual billing flow end-to-end
Sprint 25	Schools nav link, billingPeriod server persistence (migration 0021)
Sprint 26	Stripe integration (products, prices, checkout, webhooks, portal), coupon management module, payment analytics, /billing page, 141 tests
Sprint 27	Stripe products/prices created in sandbox, PayPal/ACH payment methods, CheckoutModal payment icons

28. Appendix: Sample API Requests

Validate a Coupon Code

```
POST /api/trpc/payment.validateCoupon
Content-Type: application/json
```

```
{
  "json": {
    "code": "SUMMER20",
    "planKey": "family_monthly"
  }
}
```

Response (valid coupon):

```
{
  "result": {
    "data": {
      "json": {
        "valid": true,
        "couponId": 3,
        "code": "SUMMER20",
        "discountType": "percentage",
        "discountValue": 20,
        "discountedPrice": 15.99,
        "originalPrice": 19.99,
        "savingsAmount": 4.00,
        "description": "20% off for summer",
        "isRecurring": false
      }
    }
  }
}
```

Create a Checkout Session

POST /api/trpc/payment.createCheckoutSession
Cookie: session=<jwt>
Content-Type: application/json

```
{
  "json": {
    "planKey": "family_monthly",
    "billingPeriod": "monthly",
    "couponCode": "SUMMER20",
    "origin": "https://educamp.app"
  }
}
```

Response:

```
{
  "result": {
    "data": {
      "json": {
        "checkoutUrl": "https://checkout.stripe.com/c/pay/cs_test_..."
      }
    }
  }
}
```

Submit a Demo Request

POST /api/trpc/landing.submitDemoRequest

Content-Type: application/json

```
{
  "json": {
    "fullName": "Jane Smith",
    "schoolName": "Katy ISD",
    "role": "Curriculum Director",
    "email": "jsmith@katyisd.org",
    "phone": "281-555-0100",
    "numStudents": "5000+",
    "gradeLevels": ["6-8", "9-12"],
    "subjects": ["Math", "Science"],
    "challenges": "Students are falling behind in algebra after COVID learning gaps.",
    "interestType": "District Licensing",
    "preferredTime": "Weekday Morning",
    "notes": "We are evaluating 3 platforms this quarter."
  }
}
```

Document version: Sprint 27 | Last updated: May 2026 | Maintained by: EduChamp Engineering Team